

操作系统概述

1.1 简约不简单：从 Hello World 说起

相信大家对代码片段 1.1 中的代码都不陌生。让我们将这个程序保存为 `hello.c`，并在一个操作系统上编译和运行，可以看到输出记录 1.1 中的内容。

代码片段 1.1 C 语言版本的 Hello World

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello World!\n");
6     return 0;
7 }
```

输出记录 1.1 Hello World 的编译与运行

```
# 编译 Hello World
[user@osbook ~] $ gcc hello.c -o hello

# 运行一个 Hello World 程序
[user@osbook ~] $ ./hello
Hello World!

# 同时启动两个 Hello World 程序
[user@osbook ~] $ ./hello & ./hello
[1] 144
Hello World!
Hello World!
[1]+  Done                  ./hello
```

这个过程看似非常简单，但是如果深究从键盘输入命令到屏幕输出字符整个过程中的每一个细节，就会发现事情并不简单。以下是从操作系统的角度需要思考的一些问题：

- hello 这个可执行文件是如何存储在计算机中的?
- 如何根据字符串“hello”找到存储在计算机中的 hello 文件?
- hello 文件是如何加载到内存中的? 内存的布局是什么样的?
- hello 程序在执行过程中是如何将“Hello World!”输出到显示器的?
- 两个 hello 程序是如何同时在一个 CPU 上运行的?
- 两个 hello 程序同时运行, 为什么输出的字符串不会互相覆盖?
- 是否可以支持 1 万个 hello 同时运行? 10 万个呢? 100 万个呢?
- 如果能支持 100 万个 hello 同时运行, 内存不够怎么办?
- 如果 hello 和其他应用程序同时运行, 其中有一个应用出现运行故障(如发生死循环), 是否会干扰 hello 的正常运行?

这些问题将在本书的后续章节逐步得到解答。下面, 我们将从这个简单的例子开始操作系统之旅。

1.2 什么是操作系统

Hello World 只是一个非常简单的应用, 在现实生活中, 有大量不同用途、不同复杂度的应用需要运行在操作系统之上。那么到底什么是操作系统呢? 在个人电脑领域, 我们熟悉的操作系统有 Windows、Linux、macOS 等; 在手机平台上, Android、iOS 也是操作系统。随着计算平台的变化, 还出现了很多新的操作系统概念, 例如城市操作系统、云操作系统、数据中心操作系统、智能汽车操作系统等。这些不同的操作系统有哪些共性呢? 总体而言, 操作系统有两个职责: 对硬件进行管理和抽象, 为应用提供服务并进行管理。

首先, 从硬件的角度看, 操作系统主要包含两类共性功能: 一是管理硬件, 二是对硬件进行抽象。一方面, 操作系统将复杂的、具备不同功能的硬件资源纳入统一的管理。例如, 计算机中存在多种不连续的、有限的物理内存区域, 操作系统识别这些区域的起始地址以及大小信息后, 再使用物理内存分配器进行管理。此外, 操作系统还负责管理各种外部设备(如磁盘、网卡、键盘等)的运行并处理可能的错误情况。另一方面, 操作系统负责将硬件抽象为应用程序更容易使用的资源。在这个过程中, 操作系统的核心功能是将有限的、离散的资源高效地抽象为无限的、连续的资源, 将硬件通过易用的接口提供给上层应用, 从而使应用开发者和用户无须过多考虑烦琐的硬件细节。例如, 应用开发者无须关心物理内存的硬件型号和容量大小, 而是面向一个统一的、近似无限的虚拟地址空间(如 ARM 的 AArch64 架构目前支持 2^{52} 字节即 4PB 的虚拟地址空间)进行编程; 当用户需要打印文件时, 无须考虑各种不同打印机硬件的差异, 只需点击“打印”按钮即可。

其次, 从应用的角度来看, 操作系统主要包含两类共性功能: 一是服务应用, 二是

管理应用。一方面，操作系统为应用提供了各种不同层次、不同功能的接口以提供不同类型的服务，如访问控制、应用间交互等。通过这些服务，操作系统将应用从繁杂的资源管理等系统工作中解放出来，使得 hello 应用可以通过少量的代码来实现屏幕显示的复杂功能。另一方面，操作系统也负责对应用的生命周期进行管理，包括应用的加载、启动、切换、调度、销毁等。通过对应用的管理，操作系统能够从全局角度进行资源分配，从而保证应用间的公平性、性能与安全的隔离性，防止和限制少数恶意应用对系统整体产生的影响。

由于操作系统的以上两个主要职责是分别面向硬件和应用的，因此当硬件和应用发生改变时，操作系统通常也需要随之进行调整——这正是当前的情况。一方面，当前的硬件种类越来越多。例如，智能驾驶汽车可包含通用处理器、人工智能加速器、高带宽内存、传感器、高清摄像头、毫米波雷达、激光雷达等各种硬件，操作系统需要对这些新的硬件进行管理与抽象，以便智能驾驶应用高效、方便地利用这些硬件资源。操作系统还需要支持这些硬件设备之间的相互协作，例如高清摄像头拍摄的图像需要传递给人工智能加速器进行处理。另一方面，应用的需求也越来越多样化。例如，控制类应用对时延的要求越来越苛刻，如飞行控制与智能驾驶应用需要操作系统提供实时性保障；移动应用则对低功耗的需求越来越高，如语音唤醒功能既要提供快速响应又要保持极低功耗。这些硬件和应用的新变化与新需求驱动着操作系统不断演进。

早期操作系统（如 UNIX）仅仅是操作系统内核加上一个简单的 shell。内核包含了操作系统的所有功能，通过系统调用的方式对外提供服务。shell 则以命令行的形式实现人机交互，利用操作系统的接口提供应用程序启动、切换、关闭等简单功能，如本章运行 hello 的命令行界面。

随着硬件种类和应用需求变得越来越丰富，大量共性功能沉淀到操作系统中，使操作系统的内涵与外延不断扩大。为了在稳定性与灵活性之间进行更好的权衡，操作系统在演进的过程中产生了一种新的形态：操作系统内核与操作系统框架的组合。操作系统内核通常用于实现通用的、相对稳定的功能；操作系统框架则更为灵活，可面向不同应用场景进行适配，提供更有针对性的功能。随着功能的逐步增多，操作系统框架可以进一步分为系统服务与应用框架，前者实现对操作系统内核抽象与管理能力的扩展，后者基于系统服务实现对应用开发与运行所需共性功能的增强。图 1.1 左侧展示了现代操作系统的简明结构：操作系统内核负责对硬件资源的管理与抽象，为操作系统框架提供基础的系统服务；系统服务则基于操作系统内核提供的功能进行进一步抽象与封装，从而方便上层应用框架的设计与开发；应用框架基于系统服务提供的功能，结合应用领域的特点提供应用开发与运行的共性功能，从而方便开发者开发与管理领域应用。图 1.1 右侧展示了 Android 操作系统的结构：Linux 内核提供对硬件的抽象与管理，Android 系统服务进一步扩展硬件能力，Android 应用框架为上层提供 Android 应用的开发与执行环境。

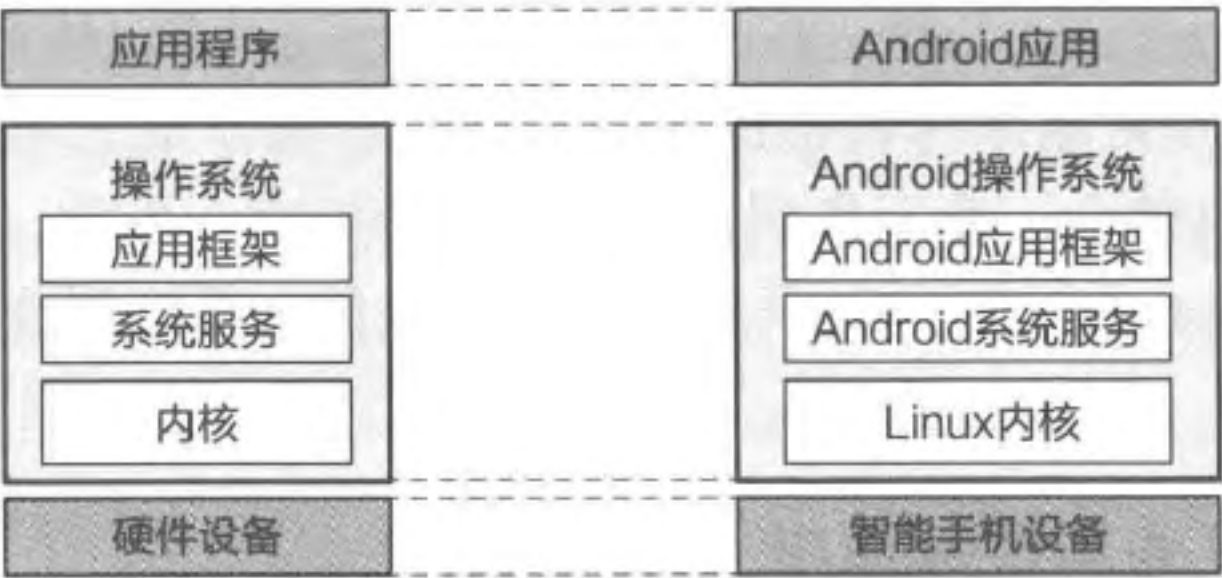


图 1.1 操作系统的简明结构

接下来，我们将沿着时间线，梳理一下操作系统的发展历史。

操作系统与应用

如果说操作系统的作用是应用管理和硬件抽象，那么如果只有一个应用，该应用直接控制硬件，是否还需要操作系统？如果应用直接和硬件打交道，直接操作和管理硬件资源，那么可以不需要操作系统。这么做的优点在于可减少由于操作系统对硬件的抽象而带来的性能损失，理论上可以 100% 发挥硬件性能。但缺点也是非常明显的：一旦硬件发生改动，应用往往也得随之改动，很难做到让这个应用在不同的硬件平台都能运行。一种可能的方法是：实现一套用于硬件资源管理与抽象的库，具有相对稳定的接口，并将其作为应用的一部分。这样当硬件发生改动时，只需要修改这个库就行了，不需要修改应用。事实上，有一些对性能和时延要求极高的场景就是通过类似的方法实现的，包括本书后面将介绍的基于外核（Exokernel）架构的库操作系统（LibOS）与虚拟化容器等方法。在这种情况下，与其说不需要操作系统，不如说操作系统的功能以库的形式存在——这也是“库操作系统”这个名字的由来。

1.3 操作系统简史

1.3.1 GM-NAA I/O：第一个（批处理）操作系统

1946 年 2 月 14 日，世界上公认的第一台通用计算机 ENIAC 在美国宾夕法尼亚大学诞生。在这之后，如何管理计算机上的程序也逐步成为一个非常重要的问题。早期的计算机采用纸带的方式记录要计算的任务，并通过打孔的纸带或磁带接收输出的结果。这样就需要一个专门的操作员值守在计算机旁边，并且操作的效率也不高。因此，人们开始关注如何自动化这一过程。

1956 年，Robert L. Patrick 和 Owen Mock 在 IBM 704 上实现了第一个实际使用的操作系统 GM-NAA I/O^[1]，即通用汽车公司和北美航空的输入 / 输出系统（General